UNIT IV

Probability Distributions

Normal Distribution:

It is a probability function used in statistics that tells about how the data values are distributed. It is the most important probability distribution function used in statistics because of its advantages in real case scenarios. For example, the height of the population, shoe size, IQ level, rolling a dice, and many more.

The normal distribution, also known as the Gaussian distribution or bell curve, is a continuous probability distribution that is symmetric around its mean, forming a **bell-shaped curve**.

Each probability distribution in R is associated with **four functions** which follow a naming convention:

1. The probability density function always begins with **'d'**.
2. The cumulative distribution function always begins with **'p'**.
3. The inverse cumulative distrobution (or quantile function) always beings with **'q'**.
4. A function that produces random variables always begins with **'r'**.

Functions To Generate Normal Distribution in R

1) dnorm()

dnorm() function in R programming measures density or Probability of a given data that follows Normal Distribution.

In statistics, it is measured by below formula-

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

where, μ -> is mean and σ -> is standard deviation.

Syntax :

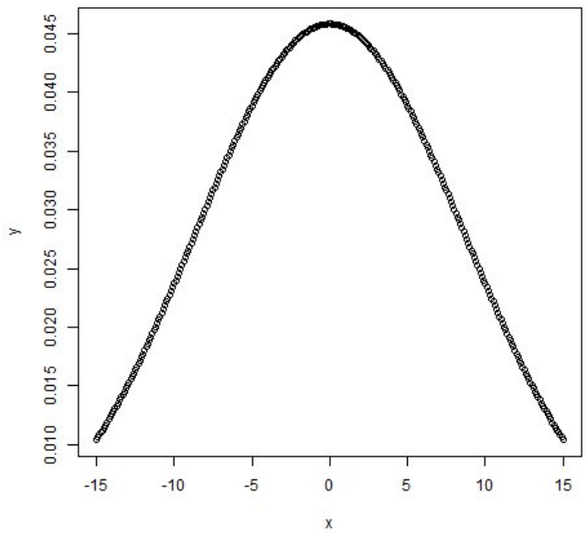
dnorm(x, mean, sd)

Example:

```
# creating a sequence of values  between -15 to 15 with a
difference of 0.1

x = seq(-15, 15, by=0.1)
y = dnorm(x, mean(x), sd(x))
# Plot the graph.
plot(x, y)
```

Output:



pnorm()
pnorm() function is the cumulative distribution function which measures the cumulative probability of a given data that follows Normal Distribution.

i.e., in statistics it is given by-

$$F_X(x) = Pr[X \leq x] = \alpha$$

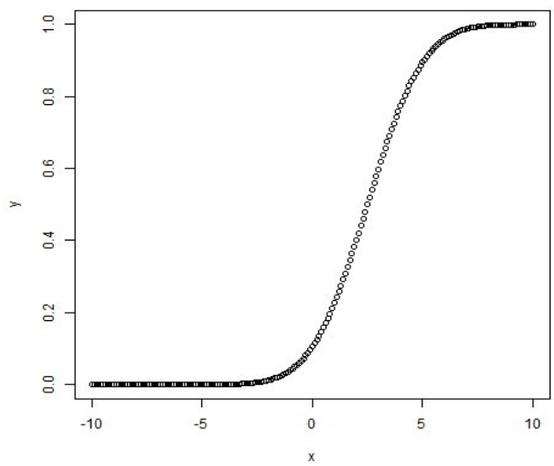
Syntax:
pnorm(x, mean, sd)

Example:

```
# creating a sequence of values between -10 to 10 with a
difference of 0.1

x <- seq(-10, 10, by=0.1)
y <- pnorm(x, mean = 2.5, sd = 2)
# Plot the graph.
plot(x, y)
```

Output :



qnorm()

qnorm() function is the inverse of pnorm() function. It takes the probability value and gives n^{th} quantile of corresponding probability value for Normal Distribution.

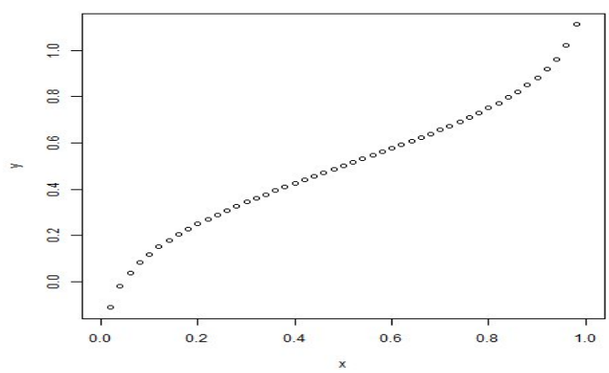
Syntax:

qnorm(p, mean, sd)

Example:

```
# Create a sequence of probability values incrementing by 0.02.
x <- seq(0, 1, by = 0.02)
y <- qnorm(x, mean(x), sd(x))
# Plot the graph.
plot(x, y)
```

Output:



rnorm()

rnorm() function in R programming is used to generate ‘n’ random numbers that follows normally distribution.

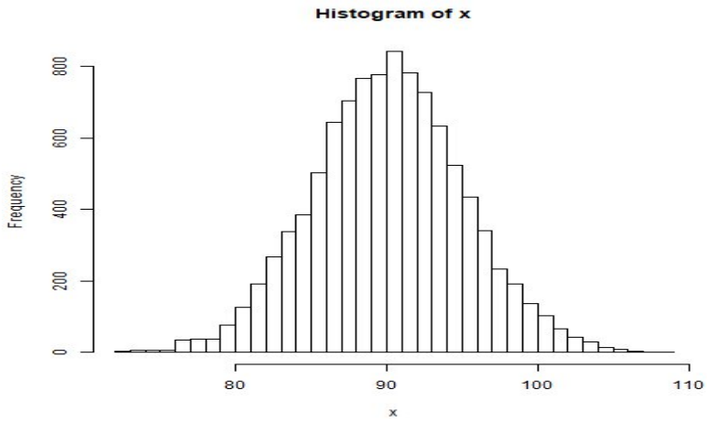
Syntax:

rnorm(x, mean, sd)

Example:

```
# Create a vector of 1000 random numbers # with mean=90 and sd=5
x <- rnorm(10000, mean=90, sd=5)
                    # Create the histogram with 50 bars
hist(x, breaks=50)
```

Output :



Binomial Distribution in R Programming

Binomial distribution in R is a probability distribution used in statistics. The binomial distribution is a discrete distribution and has only two outcomes i.e. success or failure. All its trials are independent, the probability of success remains the same and the previous outcome does not affect the next outcome.

It is also used in many real-life scenarios such as in determining whether a particular lottery ticket has won or not, whether a drug is able to cure a person or not, it can be used to determine the number of heads or tails in a finite number of tosses, for analyzing the outcome of a die, etc.

Formula:

$$P(X = k) = {}^{n}C_{r} p^{r} q^{n-r}, \text{ where } r = 0, 1, 2, 3, \ldots, n$$

p is the probability of success

q is the probability of failure

$$p + q = 1$$

Functions for Binomial Distribution

We have four functions for handling binomial distribution in R namely:

dbinorm() Function:

This function in R programming measures density or Probability of a given data that follows Binomial Distribution.

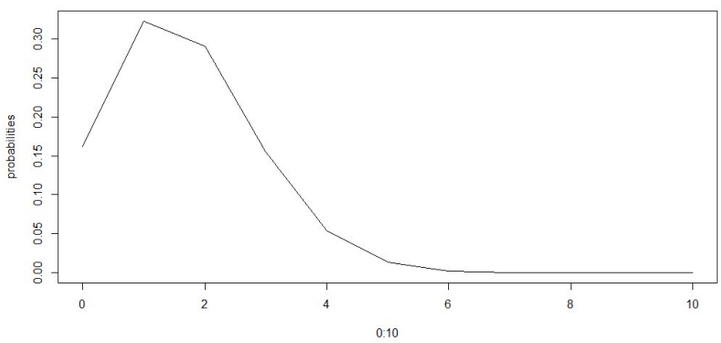
P(X = k)

Syntax:
dbinom(k, n, p)

Example:

```
dbinom(3, size = 13, prob = 1 / 6)
probabilities <- dbinom(x = c(0:10), size = 10, prob = 1 / 6)
data.frame(x, probs)
plot(0:10, probabilities, type = "l")
```

Output :
```
 [1] 0.2138454
1   1.615056e-01
2   3.230112e-01
3   2.907100e-01
4   1.550454e-01
5   5.426588e-02
6   1.302381e-02
7   2.170635e-03
8   2.480726e-04
9   1.860544e-05
10  8.269086e-07
11  1.653817e-08
```



The above piece of code first finds the probability at k=3, then it displays a data frame containing the probability distribution for k from 0 to 10 which in this case is 0 to n.

pbinom() Function
The function **pbinom()** function is the cumulative distribution function which measures the cumulative probability of a given data that follows Binomial Distribution.
ie it finds
P(X <= k)

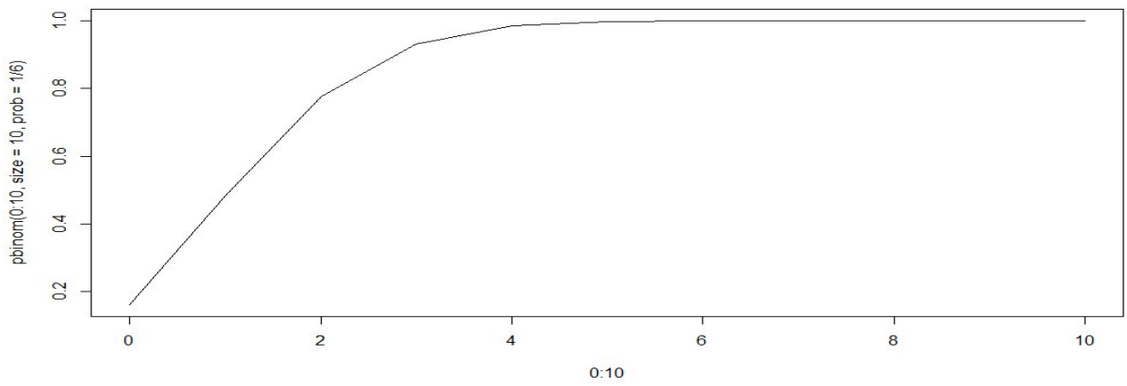
Syntax:
pbinom(k, n, p)

Example:

```
pbinom(3, size = 13, prob = 1 / 6)
plot(0:10, pbinom(0:10, size = 10, prob = 1 / 6), type = "l")
```

Output :
```
> pbinom(3, size = 13, prob = 1/6)

[1] 0.8419226
```



qbinom() Function
This function is the inverse of pbinom() function. It takes the probability value and gives nth quantile of corresponding probability value for Binomial Distribution.

Syntax:
qbinom(P, n, p)

Example:

```
qbinom(0.8419226, size = 13, prob = 1 / 6)
x <- seq(0, 1, by = 0.1)
y <- qbinom(x, size = 13, prob = 1 / 6)
plot(x, y, type = 'l')
```

Output :
> qbinom(0.8419226, size = 13, prob = 1/6)

[1] 3

- **rbinom():**
  This function in R programming is used to generate 'n' random numbers that follows Binomial distribution.
  rbinom(n, N, p)

  Where n is numbers of observations, N is the total number of trials, p is the probability of success.

Poisson Distribution:

Poisson Functions in R Programming

-

The Poisson distribution represents the probability of a provided number of cases happening in a set period of space or time if these cases happen with an identified constant mean rate (free of the period since the ultimate event). Poisson distribution has been named after Siméon Denis Poisson(French Mathematician).
Many probability distributions can be easily implemented in R language with the help of R's inbuilt functions.

There are four Poisson functions available in R:

- dpois
- ppois
- qpois
- rpois

dpois(): This function in R programming measures density or Probability of a given data that follows Poisson Distribution.

Syntax:

$dpois(k, \lambda, log)$

where,

K: *number of successful events happened in an interval*

λ *: mean per interval*

log: *If TRUE then the function returns probability in form of log*

Example:

```
dpois(2, 3)
dpois(6, 6)
```

Output:

[1] 0.2240418
[1] 0.1606231

ppois(): This function is the cumulative distribution function which measures the cumulative probability of a given data that follows Poison Distribution.

Syntax:

$ppois(q, \lambda, lower.tail, log)$

where,

K: *number of successful events happened in an interval*

λ *: mean per interval*

lower.tail: *If TRUE then left tail is considered otherwise if the FALSE right tail is considered*

log: *If TRUE then the function returns probability in form of log*

Example:

```
ppois(2, 3)
ppois(6, 6)
```

Output:
[1] 0.4231901
[1] 0.6063028

rpois(): This function in R programming is used to generate 'n' random numbers that follows Poison distribution.

Syntax:

$rpois(q, \lambda)$

where,

q: *number of random numbers needed*

mean per interval

Example:

```
rpois(2, 3)
rpois(6, 6)
```

Output:

[1] 2 3
[1] 6 7 6 10 9 4

qpois(): This function is the inverse of ppois() function. It takes the probability value and gives n^{th} quantile of corresponding probability value for Poison Distribution.

Syntax:

$qpois(q, \lambda, lower.tail, log)$

where,

K: *number of successful events happened in an interval*

λ *:mean per interval*

lower.tail: *If TRUE then left tail is considered otherwise if the FALSE right tail is considered*

log: *If TRUE then the function returns probability in form of log*

Basic Statistics:

Mean, Median, and Mode

In statistics, there are often three values that interests us:

- **Mean** - The average value
- **Median** - The middle value
- **Mode** - The most common value

Mean:

- It is the **sum** of observations **divided** by the **total number** of observations.
- It is also defined as average which is the sum divided by count.
- Mean, Median and Mode in R Programming Where, n = number of terms

Median:

- It is the middle value of the data set.
- It splits the data into two halves.
- If the number of elements in the data set is odd then the center element is median and if it is even then the median would be the average of two central elements.

- Mean, Median and Mode in R Programming Where n = number of terms

Syntax: median(x, na.rm = False)

Where, X is a vector and na.rm is used to remove missing value

Mode:

- It is the value that has the highest frequency in the given data set.
- The data set may have no mode if the frequency of all data points is the same.
- Also, we can have more than one mode if we encounter two or more data points having the same frequency.
- There is **no inbuilt function** for **finding mode in R**, so we can create our own function for finding the mode or we can use the package called modeest.
- Creating a user-defined function for finding Mode There is no in-built function for finding mode in R.
- So let's create a user-defined function that will return the mode of the data passed.
    - We will be using the table() method for this as it creates a categorical representation of data with the variable names and the frequency in the form of a table.
    - We will sort the column Age column in descending order and will return the 1 value from the sorted values.

Example: R program to find mean, mode & median. (Finding mode by sorting the column of the data frame)

```
marks <- c(97, 78, 57,78, 97, 66, 87, 64, 87, 78)
                # calculating mean
mn<-mean(marks)
        # calculating mean
med<-median(marks)
                # calculating mean
                # defining userdefined 'mode()' function
mode = function()
{
   return(names(sort(-table(marks)))[1])
}
                # call mode() function
mod<-mode()
paste("Mean=",mn)
paste("Median=",med)
paste("Mode=",mod)
```

OUTPUT

"Mean= 79.6363636363636"

"Median= 78"

"Mode= 78"

Covariance and **Correlation** are terms used in statistics to measure relationships between two random variables.

Correlation:

cor() function in R programming measures the correlation coefficient value.

Correlation is a relationship term in statistics that uses the covariance method to measure **how strongly the two vectors are related to each other**.

Syntax: cor(x, y, method)

where,

x and **y** represents the data vectors

method defines the type of method to be used to compute covariance. Default is “pearson”.

Example:

```
# Data vectors
x <- c(1, 3, 5, 10)
y <- c(2, 4, 6, 20)
            # Print correlation using different methods
print(cor(x, y))
print(cor(x, y, method = "pearson"))
print(cor(x, y, method = "kendall"))
print(cor(x, y, method = "spearman"))
```

Output:

[1] 0.9724702

[1] 0.9724702

[1] 1

[1] 1

Covariance:

In R programming, covariance can be measured using the **cov()** function. Covariance is a

Covariance is a relationship term in statistics that measure **how strongly the two vectors differ from each other**.

Syntax: cov(x, y, method)

where,

x and y represents the data vectors

method defines the type of method to be used to compute covariance. Default is “pearson”.

Example:

```
x <- c(1, 3, 5, 10)
y <- c(2, 4, 6, 20)
            # Print covariance using different methods
print(cov(x, y))
print(cov(x, y, method = "pearson"))
```

```
print(cov(x, y, method = "kendall"))
print(cov(x, y, method = "spearman"))
```

Output:

```
[1] 30.66667
[1] 30.66667
[1] 12
[1] 1.666667
```

Differences between Covariance & Correlation:

Covariance	Correlation
It shows the extent to which two variables are dependent on each other , higher the number higher the dependency.	It measures the strength of two variables considering other conditions are constant.
Range is -∞ to +∞	Range is -1 to +1 Here the symbol(- or +) signifies the direction of relation.
It is affected by scale of the variable	It is not affected by scale
It has definite units as it is derived by multiplying two numbers and their units	It is unit less and in between -1 to +1.

- Covariance indicates the direction of the linear relationship between variables.
- Correlation measures both the strength and direction of the linear relationship .

T – Test:

A hypothesis is made by the researchers about the data collected for any experiment or data set.

A hypothesis is an assumption made by the researchers that are not mandatory true.

In simple words, a hypothesis is a decision taken by the researchers based on the data of the population collected.

Hypothesis Testing in R Programming is a process of testing the hypothesis made by the researcher or to validate the hypothesis.

To perform hypothesis testing, a random sample of data from the population is taken and testing is performed.

Based on the results of the testing, the hypothesis is either selected or rejected.

There are 4 major steps in hypothesis testing:

1. **State the hypothesis**- This step is started by stating null and alternative hypothesis which is presumed as true.
2. **Formulate an analysis plan and set the criteria for decision**- In this step, a significance level of test is set. The significance level is the probability of a false rejection in a hypothesis test.

3. **Analyze sample data**- In this, a test statistic is used to formulate the statistical comparison between the sample mean and the mean of the population or standard deviation of the sample and standard deviation of the population.
4. **Interpret decision**- The value of the test statistic is used to make the decision based on the significance level.
   For example, if the significance level is set to 0.1 probability, then the sample mean less than 10% will be rejected. Otherwise, the hypothesis is retained to be true.

One Sample T-Testing :

- One sample T-Testing approach collects a huge amount of data and tests it on random samples.
- To perform T-Test in R, normally distributed data is required.
- This test is used to test the mean of the sample with the population.
- For example, the height of persons living in an area is different or identical to other persons living in other areas.

Syntax: t.test(x, mu)

Parameters: x: represents numeric vector of data

mu: represents true value of the mean

Example:

```
x <- rnorm(100)
          # One Sample T-Test
t.test(x, mu = 5)
```

Output:

```
   One Sample t-test
        data:  x
t = -49.504, df = 99, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 5
95 percent confidence interval:
 -0.1910645  0.2090349
sample estimates:
  mean of x
0.008985172
```

- Data: The dataset 'x' was used for the test.
- The determined t-value is -49.504.
- Degrees of Freedom (df): The t-test has 99 degrees of freedom.
- The p-value is 2.2e-16, which indicates that there is substantial evidence refuting the null hypothesis.
- Alternative hypothesis: The true mean is not equal to five, according to the alternative hypothesis.
- 95 percent confidence interval: (-0.1910645, 0.2090349) is the confidence interval's value. This range denotes the values that, with 95% confidence, correspond to the genuine population mean.

Two Sample T-Testing :

In two sample T-Testing, the sample vectors are compared. If var. equal = TRUE, the test assumes that the variances of both the samples are equal.

Syntax: t.test(x, y) Parameters: x and y: Numeric vectors

Example:

```
gpA<-c(18,21,24,27,30)
gpB<-c(17,20,23,26,29)
                # two sample t-test on gpA & gpB
res_ttest<-t.test(gpA,gpB)
res_ttest
```

OUTPUT

```
 Welch Two Sample t-test

data:  gpA and gpB
t = 0.33333,  df = 8,  p-value = 0.7475
alternative hypothesis: true  difference in means is not equal to 0
95 percent confidence interval:
 -5.918012    7.918012
sample estimates:
mean of x       mean of y
       24                23
```

Directional Hypothesis :

Using the directional hypothesis, the direction of the hypothesis can be specified like, if the user wants to know the sample mean is lower or greater than another mean sample of the data.

Syntax: t.test(x, mu, alternative)

Parameters: x: represents numeric vector

data mu: represents mean against which sample data has to be tested
alternative: sets the alternative hypothesis

Example:

```
x <- rnorm(100)
# Directional hypothesis testing
t.test(x, mu = 2, alternative = 'greater')
Output:
       One Sample t-test

data:  x
t = -20.708, df = 99, p-value = 1
alternative hypothesis: true mean is greater than 2
95 percent confidence interval:
 -0.2307534        Inf
sample estimates:
 mean of x  -0.0651628
```

ANOVA:

- ANOVA also known as Analysis of variance is used to investigate relations between categorical variables and continuous variables in the R Programming Language.
- It is a type of hypothesis testing for population variance.
- It enables us to assess whether observed variations in means are statistically significant or merely the result of chance by comparing the variation within groups to the variation between groups.
- The ANOVA test is frequently used in many disciplines, including business, social sciences, biology, and experimental research. Hypothesis Testing in R Programming